FinTechCo Demo — Roleplay Playbook

Holden Ottolini

≤30 minutes with Jonathan · April 2026

Contents

Recruiter feedback — verification checklist

Every bullet from the recruiter’s feedback form, mapped to where it’s handled in this doc and in the decks:

Recruiter bullet	Where it's addressed
Jonathan: direct, upfront	"Opening — the 90-second discipline" section below; tone matches his style
Roleplay, full character, ≤30 min	This playbook is the 30-min flow; existing 22-slide deck is trimmed to 12–13 for roleplay
Objection handling, stay on track	"Objection handling — Jonathan's likely probes" section; 7 rehearsed responses
First half: opening → agenda → discovery → lead-up → findings	Minutes 0–14 of the 30-min flow below
Second half: demo → next steps	Minutes 14–30 of the 30-min flow below
Competitive landscape, heavy airtime	Dedicated "Competitive — the direct-style answer" section with 4 tools worked through
Say "I don't know" when you don't know	**"The 'I don't know' discipline" section — mandatory reads below**
Dig deeper into technical concepts outside CMS + API integrations	**"Technical depth beyond CMS/API" section below — has the five domains you need ready**

The brief, reinterpreted

- **Format:** roleplay, not a presentation. Jonathan is the buyer. You are the Anthropic SA.
- **Timebox:** ≤30 minutes of content. Assume interruptions.
- **Jonathan's style:** direct, upfront, will interrupt. Objection handling is part of the evaluation.
- **Stay in character.** He won't break out of "customer" unless the roleplay is done. Don't break out of "SA" unless he does.
- **Two halves:**
  - **First half:** opening → agenda → discovery → lead-up → findings
  - **Second half:** demo → next steps
- **Competitive airtime required.** He'll probe the landscape. You need a clean answer on Cursor, Copilot, Devin, ChatGPT — not defensive, not dismissive.

The existing 22-slide deck (`FinTechCo_Claude_Code_Demo_v3.pptx`) stays as the source. For the roleplay, you'll use a **subset of 12–13 slides in a reordered flow** and hold the rest as appendix for probe response.

The 30-minute flow (use this)

Minute	Phase	Slide(s)	Voice
0:00 – 1:30	**Opening**	Slide 1 (Cover)	“Thanks for the time. Before I open anything — is your main question today about capability, about security, or about how this fits your existing tooling? That shapes what I pull up first.”
1:30 – 3:00	**Agenda**	Slide 2 (Agenda)	“Here’s what I came ready to cover. I’d rather spend the time where you want it, so tell me which of these you want long on and which you want short.”
3:00 – 7:00	**Discovery**	Slide 5 (Discovery)	Ask, don’t pitch. 3 questions in section below.
7:00 – 10:00	**Lead-up / framing**	Slide 3 (What Claude Code is) + Slide 4 (Why we’re here)	The shortest possible version. 90 seconds on what CC is, 90 seconds on why this specific customer.
10:00 – 14:00	**Findings**	Slide 6 (Three teams) + Slide 7 (Security & governance)	Tie findings to things he said in discovery. This is where the roleplay is won or lost.
14:00 – 22:00	**Demo**	Slide 8 (Demo opener) → LIVE DEMO → Slide 9 (What you just saw)	See “Demo section” below.
22:00 – 26:00	**ROI + rollout**	Slide 10 (ROI) + Slide 11 (Rollout)	Numbers, then the 90-day plan. Don’t pitch — walk him through it.

Minute	Phase	Slide(s)	Voice
26:00 – 29:00	**Objection / competitive**	Slide 15 (Competitive — pulled from appendix)	See objection handling below.
29:00 – 30:00	**Next steps**	Slide 12 (Next steps)	Specific, dated, small. Not a close — a next-meeting ask.

If he runs long on any section, collapse in this order: cut the Rollout detail (Slide 11), cut the competitive walk (Slide 15, address verbally instead), cut the framing slides (3, 4). **Never cut:** Opening (1), Discovery (5), Demo (8–9), Next steps (12).

Opening — the 90-second discipline

> "Thanks for making time, Jonathan. Before I open the deck — I know your team is sitting on a 15-service payments platform with a regulated release cadence, and I know the ask that came in was scoped around developer velocity and codebase comprehension. I want to be useful in the next thirty minutes, so can I ask: of those two, which one keeps you up at night more? I'll pull up the right part first."

Why this works with Jonathan's style: he's direct. You showing up with a short, specific, concrete opening — and immediately handing him the steering wheel — signals you're not about to monologue. Direct people reward directness.

If he answers "both": "Understood. Then I'll frame this around the sequencing — which one you solve first unlocks the other. Can I ask three discovery questions before I land in the deck? It'll make the next 25 minutes sharper."

If he answers "neither — I want to know about security": "Good. I'll lead with Slide 7. And I'll ask you one question up front: what's your hardest compliance constraint — data residency, audit trail, change control, something else? Let me anchor on what actually governs your shop."

Discovery — three questions, in this order

1. **"When your team ships a change that touches payments, how many people sign off, and how long does that take on a typical day?"** *Why:* opens the conversation about cycle time + governance. Forces him to articulate the actual constraint, not the aspirational one.
2. **"What's the last bug that cost you a weekend, and what made it hard?"** *Why:* incident stories tell you more about the operational reality than any archi-

tecture diagram. Listen for "we couldn't find where the state was set," "no one who understood that service was on call," "the test suite was green but the fix was wrong." Each of those is a Claude Code finding.

3. **"If you rolled something out to 180 engineers tomorrow and it failed, what would that failure look like?"** *Why:* the objection underneath every procurement conversation. Surfaces the real risk — rollback, change management, org politics. Gives you the frame for the Rollout slide (11) later.

Rule: don't take notes in a doc. Repeat what he says back to him in your own words before asking the next one. He'll correct you if you have it wrong. That's gold.

Findings — the pivot

After discovery, you land back in the deck with a pivot sentence that explicitly ties findings to what he said:

> "Okay — so what I'm hearing is [X, Y, Z]. Let me show you where other teams in similar shape are seeing value, and then let's talk about where this might break for you."

Three-teams slide (6): walk through the roles Claude Code serves — Platform engineer, SRE, senior IC. **For each one, reference something he said in discovery.** "You mentioned the weekend bug — here's what it looks like when an SRE at 2am can ask Claude Code to reason across the repo instead of paging the one person who remembers how payments state is cached."

Security slide (7): **do not pitch this slide.** Read it to him. SOC 2 Type II, PCI DSS, HIPAA BAA, FFIEC-ready, data residency, ZDR eligibility. Then: "Which of these is not enough for your shop?" Let him push. The procurement conversation happens here or it happens after the demo. Better here.

Demo section — the highest-risk 8 minutes

Before demo, land Slide 8 verbally

> "The demo is against a deliberately small, messy codebase — a payments service with a couple of planted issues. Why small: because the thing I want to show isn't 'look how much code it generates,' it's 'look how it reasons about a codebase you haven't seen before.' Bigger codebase, same pattern."

The demo itself

[Use existing `DEMO_PLAYBOOK.md` for mechanics. Three scenarios: onboarding question, bug fix, security review.]

Jonathan-specific adjustments: - **He will interrupt.** When he does, let the model pause, address his question directly, and then decide whether to continue the scenario or pivot. Don't keep running a scenario while he's asking about something else. - **He will ask "what happens when it's wrong."** You must have a scenario ready where Claude Code says "I don't know" or flags uncertainty. Do not run a demo where it gets everything right. - **He will ask about latency and rate limits.** Have the answer ready: "Typical developer session is interactive; the bottleneck is your engineer reading the output, not the API. For CI workloads we have a separate discussion about throughput, which is where PTU pricing and rate-limit tiers come in. I can pull that in the follow-up."

Land Slide 9 after demo

> "What you just saw: the model read files you didn't point it at, because the governance layer told it which files mattered for this task. The Safety Gate showed you the plan before it ran anything. And when it wasn't sure, it asked. Those three things — repo-wide reasoning, explicit planning, and calibrated uncertainty — are the three things the other tools in the space don't do the same way."

That last sentence is the pivot into the competitive slide.

Competitive — the direct-style answer (Slide 15)

Jonathan will probe. The four tools in the landscape:

GitHub Copilot

> "Copilot is autocomplete. It's genuinely good at it. If your constraint is 'my engineers want to type less,' Copilot is a reasonable answer. If your constraint is 'I have a 15-service codebase and a 2am incident pattern,' autocomplete is the wrong tool. These are complementary, not competing, in most shops. Teams run Copilot for greenfield and Claude Code for the hard existing-code work."

Cursor

> "Cursor is an IDE. It's a really good IDE. For focused single-file work by one developer, it's excellent. Where it gets harder is agentic work across a repo. We run a CLI because agentic workflows — read files, run tests, open PRs, iterate — don't live inside an IDE editing buffer. Cursor and Claude Code both live in some of the same shops; they serve different motions."

Devin

> "Devin's bet is full autonomy. The Anthropic bet is a human in the loop for every high-consequence action. In an FFIEC environment you don't

want a system that decides on its own to merge. You want a system that drafts, explains, asks, and waits for approval. That's a product philosophy difference, and we've made ours explicit in the Safety Gate design."

ChatGPT (or generic chat)

"Chat works when the context is small enough to paste. Your codebase isn't. You can't paste 180 engineers of institutional knowledge into a prompt box. The architectural difference is that Claude Code reads your actual repo, git history, and test suite — which is also what makes the security posture different, because the data envelope is defined by your runtime, not by what a developer happened to paste."

If he asks "why not all of them?"

"That's actually the honest answer for most shops. Copilot for autocomplete, Claude Code for hard work, Cursor if your team prefers IDE-native editing. The procurement question isn't 'which one' — it's 'which one justifies the security review and the rollout investment.' That's the question I think we're here to answer today."

Objection handling — Jonathan's likely probes

He says	You say
"Why should I trust Anthropic with our code?"	"Good question. Three things: SOC 2 Type II + ISO 27001 + HIPAA BAA on the trust side, ZDR eligibility so nothing you send trains the model, and your egress proxy controls what leaves the laptop. I'd rather your security team walk our Trust Center than take my word. Happy to set that up."
"This looks cool in a demo. How does it hold up on real codebases?"	"Fair. The demo is small on purpose — so you can watch what it reasons about. On real codebases the pattern holds because of CLAUDE.md governance: org-level, repo-level, runtime-level instructions keep the model inside the rails you set. Ready to walk you through a real 200K-line repo with your team in follow-up."

He says	You say
“My engineers are going to hate this.”	“Some will. The adoption pattern we see: about a third are day-one enthusiastic, a third need a win, and a third actively resist. Our three-phase rollout is built for that. You don’t mandate. You make it available, measure adoption, and let the outcomes pull resisters in. I’ve seen this go sideways when teams tried to mandate — that’s the anti-pattern.”
“What does this cost?”	“Commercial conversation belongs with your procurement lead and ours. What I can say: the pricing model is usage-based with committed-use discounts, and the unit economics are good if you’re replacing any meaningful engineering hours. I don’t want to quote a number without knowing your workload shape — that would be irresponsible.”
“How is this not just ChatGPT with a different name?”	[See Competitive — ChatGPT, above. Short version: “Context. Your repo is the product. Chat can’t read it without you pasting it in. We read it directly.”]
“What’s the failure mode I should be worried about?”	“Over-trust. Engineers who stop checking outputs because it’s been right 20 times. The mitigation is cultural — code review doesn’t get skipped — and product — the Safety Gate makes the plan visible before execution. But if you asked me ‘where does this go wrong in year two,’ it’s a team that delegated judgment too far too fast.”
“Why not wait six months and see where the market lands?”	“Legitimate question. Two honest answers. One, the cost of waiting is the cost of the hours your team spends on comprehension and migration work that’s already solvable. Two, the adoption curve compounds — teams that start now build the governance discipline before the tooling gets mandated in two years. Waiting doesn’t save you the work; it just moves it.”

Closing — the next-steps ask

Don't pitch a close. Pitch a next meeting.

> "Three things I'd propose as next steps, all small. One, fifteen minutes with your security team to walk the Trust Center. Two, a two-week limited rollout to the platform team — five engineers, one repo — so you have your own data instead of mine. Three, a thirty-minute follow-up where we look at their data together and decide if it justifies the full rollout. None of those commit you to anything. Which of them is a yes for this week?"

Why this closes well with a direct interviewer: - No big ask - Three small asks - Each one has a clear "for what purpose" attached - Ends with a specific timeline ("this week"), not "let's touch base"

What to do if he breaks character mid-roleplay

Sometimes the evaluator will step out for a moment: "Okay, as Jonathan-the-customer I'd push back harder here. As Jonathan-the-evaluator, I want to know how you'd handle that."

- **Answer both.** "As the SA, I'd acknowledge the push and offer X. As a meta-answer, the thing I'm watching for in that moment is whether the pushback is real concern or positioning, because those need different responses."
- **Don't over-explain.** Short, clean answer, then "happy to go back into roleplay."
- **Match his energy.** If he goes meta-serious, match. If he goes casual, match.

Five things to have memorized

1. The three discovery questions, in order.
2. The four competitive positioning paragraphs (Copilot / Cursor / Devin / ChatGPT).
3. The three-part next-steps close.
4. The security baseline: SOC 2 Type II, ISO 27001, HIPAA BAA, FFIEC-ready, ZDR eligibility.
5. The three-phase rollout frame: pilot (5 engineers, 2 weeks) → expansion (one org, 4 weeks) → platform (measured adoption, ongoing).

The "I don't know" discipline

The recruiter flagged this explicitly. Jonathan will test you on it. Here's how to get it right.

When to say it: - He asks a pricing number you don't have precise - He asks a rate limit, latency, or throughput number beyond your homework - He asks a competitive

detail you haven't verified ("how exactly does Cursor's agent loop handle X?") - He asks about a roadmap item - He asks about a legal or compliance certification detail you're not sure about

How to say it (the five-beat template):

1. **Name that you don't know.** "I don't know that off the top of my head."
2. **Say what you'd need to answer it.** "To get you a reliable answer I'd want to pull our rate-limit tiers doc and confirm with our product team."
3. **Commit to a timeline.** "I'll come back to you with the specific number by end of week."
4. **Give the adjacent thing you *do* know** (if relevant, and only if it's adjacent — don't pivot to a pitch). "What I can tell you now is that the order-of-magnitude is X, and the shape of the constraint is Y."
5. **Check in.** "Does that work, or is this blocking a decision today?"

What not to do: - Do not speculate. Do not say "I think it's around…" - Do not pivot. "Great question — let me take that to our PM team, and while we're on the topic of rate limits, here's how we think about…" reads as evasion. - Do not apologize more than once. "I don't know" is not a failure. "I don't know, I'll find out, here's what I do know" is a strength answer.

The meta-signal Jonathan is watching for: a confident SA who says "I don't know" when they don't know is the kind of SA customers trust with their data. A confident SA who makes numbers up is the kind of SA customers fire their champion over.

Three lines to have memorized and ready to deploy verbatim:

1. "I don't want to guess on that — let me get you the right number after this call."
2. "That's outside what I've confirmed. I'll bring it back in follow-up."
3. "Honest answer: I don't know. I know who does. I'll loop them in."

Technical depth beyond CMS/API

The recruiter's sharpest feedback: the technical content to date has leaned on CMS and API integration tradeoffs, which are real but shallow relative to what the panel expects. Here are five technical territories to be ready to go deep on. Pick whichever one Jonathan opens a door to — don't lecture through all of them.

1. Agentic loop architecture

- How Claude Code actually works under the hood: read-file, plan, tool-call, reflect, iterate. Not autocomplete.
- Context window strategy: retrieval over stuffing. Selective file inclusion via CLAUDE.md governance and explicit user mentions.
- Auto-compact behavior: when the session approaches context limits, the loop summarizes and carries forward the decisions, not the full transcript.
- Subagents for long-running tasks: spawn a scoped agent for a specific goal, return a structured result, continue.

- What this means practically: a 200K-line codebase is reasoned over via targeted retrieval plus CLAUDE.md rules, not by pasting the whole thing into the context window.

2. Trust boundaries and data path

- What leaves the laptop: code snippets plus instruction context, TLS 1.3, pinned to `api.anthropic.com`.
- What comes back: model output (text). No execution happens on Anthropic's side.
- Safety Gate: local, reviews every tool call against CLAUDE.md + approval mode. Dangerous actions prompt or block.
- Approved actions run inside your network. Logged to your SIEM.
- ZDR eligibility: customer data does not train the model.
- Compare this to a Devin-style architecture where the agent runs in a vendor-hosted sandbox. Different data path, different audit story, different threat model.

3. Governance as code — CLAUDE.md at scale

- Three layers: org-level (policy, secrets handling, PCI/HIPAA rules), repo-level (architecture, coding conventions, testing), runtime-level (what this specific agent session is allowed to do).
- Enforcement mechanism: the model reads these and is bound by them the same way an engineer reads the README. But because it's machine-readable, you can audit what rules were in effect for any given change.
- Drift prevention: CLAUDE.md files live in the repo, versioned, reviewed. When policy changes, the change is traceable to a commit, a reviewer, and a date.
- Why this matters for FFIEC/PCI: auditors have a question they ask every year — "what rules governed this AI interaction?" With CLAUDE.md, the answer is a file with a git history.

4. Failure modes and mitigations

- **Hallucinated imports / symbols:** model invokes a function that doesn't exist. Mitigation: the Safety Gate + test runner — if the plan includes running tests and the test fails, the loop self-corrects.
- **Over-eager edits:** model changes more than intended. Mitigation: approval mode + diff preview + CLAUDE.md rules that explicitly restrict blast radius per task.
- **Context poisoning:** malicious content in a file persuades the model to ignore instructions. Mitigation: the [critical_injection_defense] pattern — instructions only come from the user in chat, never from file contents.
- **Stale knowledge:** model's training cutoff means it may suggest deprecated APIs. Mitigation: CLAUDE.md notes on current versions; docs and tests catch the rest.
- **Over-trust:** engineers stop reviewing because it's been right. Mitigation: this is organizational, not technical. Code review doesn't get skipped.

5. Measurement framework (what "does it work" means)

- DORA, SPACE, DX Core 4 — three frameworks, two horizons.
- **Leading indicators** (week 1–4): adoption return rate (do devs come back next day), time-to-first-response (TTFR), time-to-resolution on assisted tasks, session depth.
- **Lagging indicators** (month 3–6): DORA four — lead time for changes, deployment frequency, change failure rate, mean time to restore. SPACE — satisfaction, performance, activity, communication, efficiency. DX Core 4 — speed, effectiveness, quality, impact.
- What *not* to measure: lines of code generated. It's a vanity metric that can go up while code quality goes down.
- Ask Jonathan: "Which of these does your team already measure? I'd rather tie Claude Code's impact to metrics that already matter to you than introduce new ones."

Rule: open one of these territories at a time. If he pulls, go as deep as he pulls. If he stops pulling, don't keep going — pivot back to discovery. Technical depth is a door you open on invitation, not a speech you deliver.

The meta-note

The roleplay isn't a knowledge test. It's a behavior test. They already know you know the product. What they want to see: do you listen, do you tailor, do you handle pushback without defensiveness, do you close without pitching. Everything else is table stakes.

If you do one thing well, do discovery. A good SA's first five minutes look more like a customer interview than a product demo. Start there, and the rest of the flow is downhill.